

Data Structures and Algorithms
-00940224-

- **Student:** [Suheil Khourieh](#) - [REDACTED]

Question 1:

Prove that no general data structure exists (without relying on the keys of the elements but for comparison purposes) that supports the following operations:

- Inserting a new element in $O(\sqrt{\log(n)})$.
- Deleting an element based on its key in $O(\log(n))$.
- Searching for an element based on its key in $O(\log(n))$.
- Extracting the minimal element and returning it in $O(\sqrt{\log(n)})$.

In all the methods, n represents the current number of elements in the data structure.

Solution:

Before proving anything, we know (based on a theorem that was proven in the lectures) that "The run-time of a comparison based sorting algorithm must be $\Omega(n \log(n))$."

Assume by contradiction that such data structure exists (Supports all the methods mentioned above in the given run-time), if such a data structure exists, let's introduce the a new comparison based sorting algorithm that takes as an input an array of element, where the keys of the elements support the basic comparing queries. The algorithm sorts the input array by inserting the elements one by one into the data structure defined in the question, upon completion, the algorithm keeps on extracting the minimal element and adds it to a new array by order, resulting in an ordered version of the given array, thus sorting the array. Here is the pseudo-code:

```

1  function Sort(Elemnts[] arr) {
2      DS = new givenDS
3      for i = 1,...,n do
4          DS.insert(arr[i]) //O(sqrt(logn))
5      end
6      sorted = new Elemnts[n]
7      for i = 1,...,n, do
8          sorted[i] = DS.extract_min() //O(sqrt(logn))
9      end
10     return sorted
11 }
```

We can easily see that pseudocode above indeed describes a sorting algorithm that relies on a data structure that supports the provided methods.

The run-time of the algorithm is $O(n\sqrt{\log(n)})$:

Line 2 creates a new instance of the data structure, doing so is not related to the input array nor it is related to its size, thus, doing so takes $O(1)$.

The loop lines 3-5 loops over all the elements in the input array (n iterations) and in each iteration it inserts the element in the data structure, each insertion operation takes $O(\sqrt{\log(n)})$ according to the question, resulting the loop lines 3-4 in taking a total of $O(n\sqrt{\log(n)})$.

Line 6 creates a new empty array that will hold the elements in the sorted way, creating an empty array is $O(1)$.

The loop lines 7-9 fills the new array by removing the minimum value from the data structure and adding it to the array, inserting a new element to an array takes $O(1)$,

extracting the minimum element from the data structure takes $O(\sqrt{\log(n)})$, thus each iteration takes $O(\sqrt{\log(n)})$, the loop loops n times resulting in a total run-time of $O(n\sqrt{\log(n)})$ for the second loop.

In total the algorithm takes $O(n\sqrt{\log(n)})$. This contradicts the proven theorem on comparison-based sorting that was introduced in the lectures that states that every comparison-based sorting algorithm's run-time is $\Omega(n\log(n))$, but clearly the algorithm that we found contradicts this fact. $\Rightarrow$ No such data structure can exist! ■.

Question 2:

Find an upper asymptotic bound, as tight as possible, for the following recurrence relation and prove using induction. Assume that every recurrence relation holds: $T(1) = O(1)$.

Part 1: $T(n) = T(n - 1) + n$:

Solution:

Since $T(n)$ is a function that is related to $T(n - 1)$, we can replace $T(n - 1)$ with its appropriate value that is related to $T(n - 2)$ and so on until we get a general equation:

- Develop:

$$\begin{aligned}
 T(n) &= T(n - 1) + n = \\
 &= T(n - 2) + n - 1 + n = T(n - 2) + 2n - 1 = \\
 &= T(n - 3) + n - 2 + 2n - 1 = T(n - 3) + 3n - 1 = \\
 &= \dots = \\
 &= T(n - j) + j \cdot n + \sum_{i=0}^{j-1} i
 \end{aligned}$$

- Setting $j = n - 1$, the halt condition is met:

$$T(n) = T(n - (n - 1)) + (n - 1) \cdot n + \sum_{i=0}^{(n-1)-1} i = T(1) + n^2 - n + \sum_{i=0}^{n-2} i$$

$\sum_{i=0}^{n-2} i$ is the sum of an arithmetic series, thus $\sum_{i=0}^{n-2} i = \frac{(n-2) \cdot (n-1)}{2}$.

$$\Rightarrow T(n) = T(1) + n^2 - n + \frac{(n-2) \cdot (n-1)}{2} = T(1) + n^2 - n + \frac{n^2 - 3n + 2}{2}$$

$$\Rightarrow T(n) = T(1) + \frac{3n^2}{2} - \frac{5n}{2} + 1$$

Since $T(1) = O(1)$ and because we focus on the dominant term, that is $\frac{3n^2}{2}$:

$$\Rightarrow T(n) = O(n^2)$$

- Verify by induction that $T(n) \leq c \cdot n^2$ for some constant c

Base: $n = 1$

$$T(1) = O(1) \leq c \cdot 1^2 = c$$

Hypothesis: Assume that $T(m) \leq c \cdot m^2$, $\forall 0 < m < n$

Step: Proof for n

We need to prove that $T(n) \leq c \cdot n^2$, we know that $T(n) = T(n - 1) + n$.

According to the induction hypothesis, by setting $m = n - 1$ we get

$$T(m) = T(n - 1) \leq c \cdot (n - 1)^2$$

$$T(n) = T(n - 1) + n \leq c \cdot (n - 1)^2 + n = cn^2 - 2cn - c + n = cn^2 + (1 - 2c)n - c$$

For a large enough c , we can ensure that the term $1 - 2c \leq 0$, resulting in $T(n) \leq n^2$.

Part 2: $T(n) = 2T\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) + n$:

Solution:

The constant 17 in the recurrence relation is insignificant, this is true because each time the size is “cut” in half, adding 17 each time does not have a strong effect on the time-complexity for large n , in addition to that, by expanding the recurrence function, we see that each time the 17 is cut in half, resulting to it being close to zero and having almost no effect on the result. Thus, we can discard it and get the following:

$$T(n) \approx 2T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + n$$

$$\text{Let } a = 2, b = 2, f(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n) \Rightarrow T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n).$$

According to the master theorem, this is case 2, thus

$$T(n) = \Theta(n^{\log_2 2} \cdot \log(n)) = \Theta(n \cdot \log(n)) \Rightarrow T(n) = O(n \cdot \log(n))$$

- Verify by induction that $T(n) \leq c_1 n \cdot \log(n)$ for some constant c_1

Base: $n = 1$

$$T(1) = O(1) \leq c_1 \cdot 1 \cdot \log(1) = 0$$

Hypothesis: Assume that $T\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \leq c_1 \cdot \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right)$

Step: Proof that $T(n) \leq c_1 \cdot n \cdot \log(n)$

We know that $T(n) = 2 \cdot T\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) + n$, according to the induction hypothesis, we

$$\text{get } T(n) = 2 \cdot T\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) + n \leq 2 \cdot \left(c_1 \cdot \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right)\right) + n =$$

$$2c_1 \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) = 2c_1 \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot \log\left(\left\lfloor \frac{n+34}{2} \right\rfloor\right) \leq$$

$$2c_1 \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot (\log(n + 34) - \log(2))$$

Let $a = 2c_1$, $b = \log(2)$, a and b are constants:

$$\Rightarrow 2c_1 \left(\left\lfloor \frac{n}{2} \right\rfloor + 17\right) \cdot (\log(n + 34) - \log(2)) = (c_1 n + a) \cdot (\log(n + 34) - b) \leq$$

$$c_1 n \cdot \log(n) \blacksquare$$

Part 3: $T(n) = 4T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O(n)$

Solution:

Let $a = 4, b = 2, f(n) = O(n^{\log_b a - \epsilon}) = O(n^{\log_2 4 - \epsilon}) = O(n^{2 - \epsilon})$.

Let $\epsilon = 1$, we get $\Rightarrow f(n) = O(n^{2-1}) = O(n)$.

We can easily see that $T(n) = a \cdot T\left(\left\lfloor \frac{n}{b} \right\rfloor\right) + f(n)$, where a, b and $f(n)$ are as mentioned above. According to the master theorem, specifically case 1, we get that $T(n) = \Theta(n^{\log_b a}) = \Theta(n^{\log_2 4}) = \Theta(n^2) \Rightarrow T(n) = O(n^2)$

- Verify by induction that $T(n) \leq c_1 \cdot n^2$ for some constant c_1

Base: $n = 1$

$$T(1) = O(1)$$

Hypothesis: Assume that $T\left(\left\lfloor \frac{n}{4} \right\rfloor\right) \leq c_1 \left(\left\lfloor \frac{n}{4} \right\rfloor\right)^2$

Step: Proof that $T(n) \leq c_1 n^2$

We know that $T(n) = 4T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + O(n) \leq 4T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + c_2 n$. According to the induction hypothesis, we get: $T(n) \leq 4c_1 \left(\left\lfloor \frac{n}{2} \right\rfloor\right)^2 + c_2 n = 2c_1 n^2 + c_2 n = O(n^2)$. ■

Question 3:

Let $L = \{1, \dots, k\}$ where k is some natural number. In the context of this question, the elements of the set L are letters. A finite set of letters from L is called a string. Let L^* be the set of all string. (The set L^* is infinite, even though the set L is finite).

Given a string $w \in L^*$, mark the j^{th} letter in w with $w(j)$ where $1 \leq j \leq |w|$. We say that the string $w \in L^*$ is lexicographically smaller than the string $w' \in L^*$ if there exists $0 \leq j \leq |w|$ where $w(i) = w'(i)$ for all $1 \leq i \leq j$ and one of the following conditions is met:

- $|w| = j < |w'|$ (The length of w is j and the length w' is greater than j); or
- $w(j+1) < w'(j+1)$.

In this case, we write $w < w'$.

This section of the question deals with a data structure for elements that their keys are strings, sorted in lexicographic order (Note that this is a strong full order). Assumptions:

- Uniqueness of the keys.
- The following operations take a constant run-time: clarifying the length of a given string, accessing (reading and writing) to different letters in the given string, lexicographical comparison between two given strings, copying a string and checking if a string is a prefix of another.
- There exists strings $w_{-\infty}$ and w_{∞} that hold $w_{-\infty} < w < w_{\infty}$ for every string w that we might meet.

Program a data structure D for elements that their keys are strings, that support the following operations:

- $Init(D)$, initializes a new data structure empty of elements.
- $Insert(D, x)$, that takes in addition to D , a pointer to a new element x and inserts it to D .
- $Delete(D, w)$, that takes in addition to D , a string w and deletes from D the elements whose key is w if such element exists in D .
- $CountPrefix(D, x)$, that takes in addition to D , a string w and returns the number of elements in D where w is a prefix to their keys.

The run-time of $Init(D)$ must be a constant, the run-time of all the other operations should be $O(\log(n))$ and the storage that D uses must be $O(n)$ where n represents the current number of elements in D .

Example: let's assume that $L = \{1,2,3\}$ and that in D the following keys are stored: $\{223,3,123,11234,2223,22,111111,22332\}$. According to the definition of lexicographic order: $111111 < 11234 < 123 < 22 < 2223 < 223 < 22332 < 3$. Running $CountPrefix(D, 223)$ on the above example will return 2, since the string 223 constitutes a prefix to two keys in D , 223 and 22332. Explain the way the data structure works. You should provide accurate explanations to the following questions:

- What are the different attributes with the different types that are tied to the requested data structure?
- How are the objects of those type stored together?
- How is the method $CountPrefix(D, w)$ implemented?
- You should provide a brief explanation on the implementation and the run-time of the rest of the operations.

Solution:

We will suggest implementing and storing the elements using the known data structure *Two3Tree*, we decided to use such data structure for its efficient *Init*, *Insert* and *Delete* operations. We will make some adjustments to the data structure without affecting the run-time complexity of these operations.

Before explaining the changes and the way the data structure will be implemented, we will need to clarify some points:

The nodes that are used to assemble the data structure will be the regular nodes we were taught in the lectures with adding an attribute *size* that describes the total number of leaves that are in the sub-tree rooted from the node, this is useful for implementing the *rank* method that will be responsible to finding the order statistic of a leaf for implementing the method *CountPrefix*.

The implementation of the methods needed for assembling the tree will remain almost the same, the only difference is that the sentinels now will be $w_{-\infty}$ and w_{∞} . The comparing query $==$ will be used and returns true if and only if both keys are exactly the same.

The comparing query $\leq$ will be replaced with $<$ and $==$, for example:

$$w_1 \leq w_2 \Rightarrow w_1 < w_2 \text{ OR } w_1 == w_2$$

Before proceeding with the implementations of the necessary operations, we will need to prove the following lemma:

Lemma:

$$w_1 \text{ is a prefix of } w_2 \Rightarrow w_1 < w_2 \text{ or } w_1 \equiv w_2$$

Proof:

Let $w_1, w_2 \in L^*$, be two strings, mark the length of w_1 with j .

We know that w_1 is a prefix of w_2 , if the length of w_2 is also j then $w_1 \equiv w_2$. This is true according to the definition of prefix.

Definition of prefix:

$$w_1 \text{ is a prefix of } w_2 \Leftrightarrow w_1(j) = w_2(j), \forall j = 1, \dots, |w_1|$$

If the length of w_2 is greater than j , let it be marked with d , and because

w_1 is a prefix of w_2 , we know that the first j letters are the same for both strings.

According to the definition of lexicographically smaller ($w_1 < w_2$), if the letters of two strings are equal starting from the first letter to the ending letter of w_1 and the length of w_1 is smaller than the length of w_2 then $w_1 < w_2$.

If the length of w_1 is greater than the length of w_2 then w_1 can't be a prefix of w_2 according to the definition of prefix. ■

$$\Rightarrow w_1 \text{ is a prefix of } w_2 \Rightarrow w_1 < w_2 \text{ or } w_1 \equiv w_2$$

Init(D):

The *init(D)* method will be slightly changed; the sentinels will be $w_{-\infty}, w_{\infty}$.

The run-time of the method will be the same as before; $O(1)$.

Insert(D, x):

As mentioned before, insert will go through a slight change; replacing $\leq$ with ($<$ or $==$), the $\neq$ query will remain untouched.

The implementation of *Insert* requires some additional helping methods such as *Insert_{AndSplit}*, *Update_{key}* or *Set_{Children}*. Those methods will also be implemented with the same change mentioned above.

The run-time of the method will be the same as before; $O(\log(n))$

Delete(D, x):

In a similar way to the *insert* method, the $\leq$ query will be replaced with ($<$ or $==$) and exactly as before, all the helping methods that were required for the implementation of the original tree will also be implemented while applying the same changes, hence the run-time will remain unchanged; $O(\log(n))$.

Rank(x):

We will add the method *rank*(x) to the implementation. The *rank*(x) method takes as an input a leaf and returns its order statistic in the tree. We saw the implementation for this method in the tutorials. In our case, just like before we will need to adjust the comparison queries. *rank*(x) will remain $O(\log(n))$.

Search(x, k):

We will need to implement the *search* method, it will go through the same changes done to the previous methods, in addition to that change, in case the key is not found, *search* will now return the leaf it reached to, making it the smallest node that its key is larger than k . $O(\log(n))$

searchBigger(x, key):

We will introduce a new method; let it be called *searchBigger*(x, key).

searchBigger method will work in a very similar way to the known *search* method, but instead of searching for a key, it will search for the first key that is "larger" than key and that *key is not a prefix of it*.

This can easily be implemented by adding and altering the ifs done, the changes will force the search to dive into the sub-tree that its root's key is larger than the key we are searching for and then returning the first leaf that its key is larger than the key we are looking for and that the key is not a prefix of. $O(\log(n))$

CountPrefix(D, w):

The way *CountPrefix* will be implemented is very simple, it will first call the method *search*($D.root, w$), the result will be our lower leaf, then it will call the method *searchBigger*($D.root, w$), the result will be our upper leaf.

Now the following checks are done:

If w **is not** a prefix of the lower leaf or the lower leaf is the right sentinel we will return 0, indicating that w is not a prefix of any string in D .

If w **is not** a prefix of the upper bound, then our new upper bound is the predecessor of our current upper bound, this can be achieved using the known predecessor method that works in $O(\log(n))$.

When done, and after making sure that our lower and upper bounds are actually what they should be, we will return the rank of the upper bound – the rank of the lower bound + 1.

Because the method is implemented using a calling fixed time $O(\log(n))$ operations, its run-time also is $O(\log(n))$.

Question 3

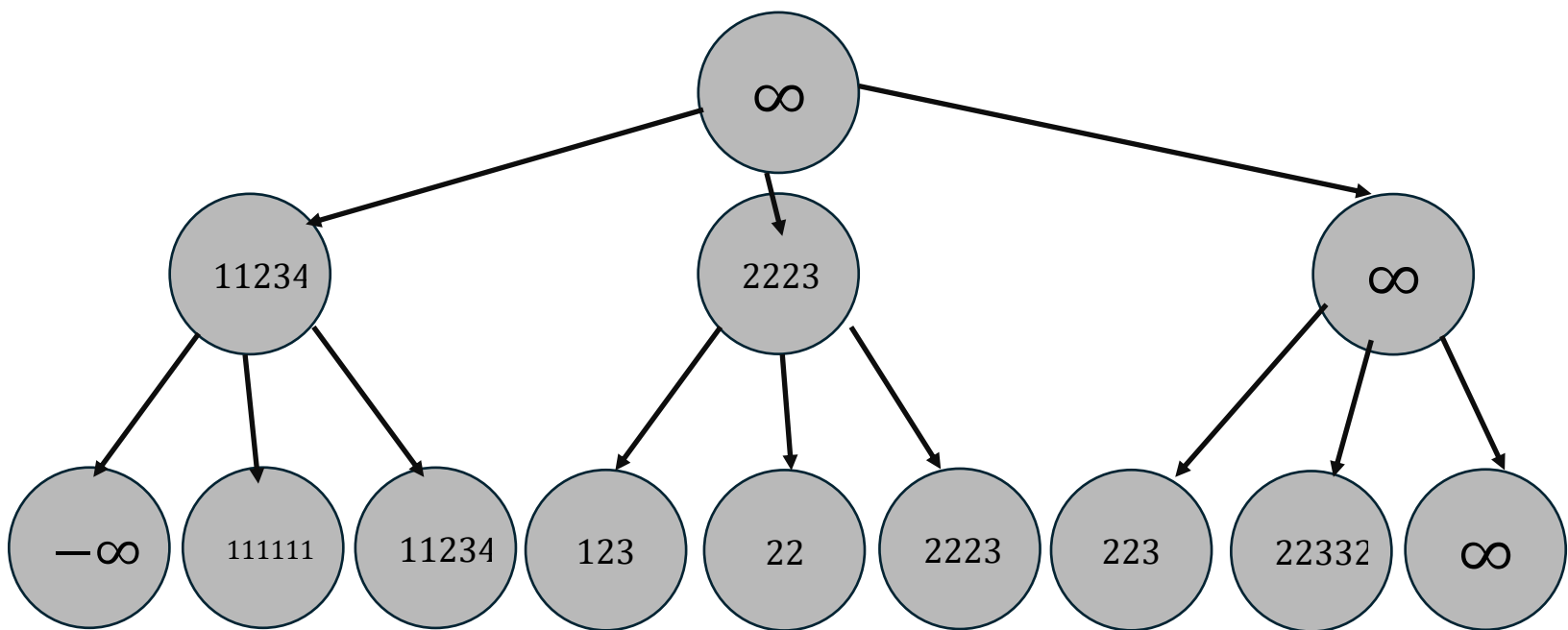
-00940224-

Due date: 23.1.25

This is how the tree will look like if we are storing the strings from the example.

The lower leaf will be 223 and the upper leaf will be 22332.

$$\text{Rank}(22332) = 7, \text{Rank}(223) = 6 \Rightarrow 7 - 6 + 1 = 2$$



Question 4:**Part 1:**

Given a unique set of keys S , we define $max - min$ print of S to be the printing of the maximal key in S , then the minimal key in S , then the second largest key in S , then the second smallest and so on. Suggest an algorithm that takes as an input a binary search tree T that contains n unique keys and prints the stored keys in a $max - min$ print. The run-time complexity of the suggested algorithm should be $O(n)$. You should provide a detailed explanation on how the algorithm works and to its correctness. There is no obligation to provide pseudo-code.

Solution:

There is no assumptions on the given binary tree, hence the height of the tree can be as big as n where n is the total number of keys stored in the tree, thus taking advantage of known operations such as $Tree_Minimum(x)$, $Tree_Delete(x)$ and similarly $Tree_Maximum(x)$ that was introduced in the lectures is not the ideal approach to the question, thus we will suggest a more time efficient approach: Based on the properties of the binary search tree (Every internal node's key is larger than it's left child and smaller than it's right child) we can traverse the tree resulting in a sorted sequence of the keys/elements, such order can be achieved using the known in-order traversal. To store the keys/elements of the tree according to the in-order traversal, we will need to use a data structure to store the keys. An array is a good data structure for this purpose, but since the tree size is unknown, we will need to introduce a new method that finds the size of the tree rooted from x :

```

1  function size(x){
2      if (x = NIL) {
3          return 0
4      }
5      return 1 + size(x.left) + size(x.right)
6  }

```

This method clearly serves the point, the size of any tree rooted from x is:

$$1 + \text{size}(\text{sub tree rooted from the left child of } x) + \text{size}(\text{sub tree rooted from the right child of } x)$$

The run-time complexity of this method is clearly $O(n)$, this can be easily proven:

$T_{\text{size}}(n) = T_{\text{size}}(m) + T_{\text{size}}(n - m) + O(1)$ where n is the size of the tree rooted from x , m is the size of the tree rooted from $x.\text{left}$ and $n - m$ is the size of the tree rooted from $x.\text{right}$.

Now that we can calculate the size of any binary search tree in $O(n)$, we can proceed with the solution to our problem, we will suggest an algorithm that traverses over the tree in-order and fills an array accordingly, resulting in an ordered versions of the keys:

```

10 function traversal_save(x, arr, idx){
11     if(x == NIL){
12         return idx
13     }
14     i = traversal_save(x.left, arr, idx)
15     arr[i] = x.key
16     j = traversal_save(x.right, arr, i + 1)
17     return j
18 }

```

The above pseudo-code describes the process of saving the keys of a binary search

tree in an array, the result is the ordered version of the keys, the method is very similar to the known *in_order* traversal, some changes are done to maintain an index for indicating the insertion placement of the next key in the array.

The run-time complexity of this method is also $O(n)$.

```

20 function max_min(T){
21     size = size(T.root)
22     arr = new arr[size]
23     traversal_save(T.root, arr, 1) // 1 is passed instead of 0 because the first element of the array is arr[1]
24     for i = 1,...,(size/2) do
25         if (i == size - i + 1){
26             print(arr[i])
27             break
28         }
29         if (i > size - i + 1){
30             break
31         }
32         print(arr[size - i + 1])
33         print(arr[i])
34     end
35 }

```

max_min(T), clearly does the intended.

Correctness:

Given a binary search tree T , the method first finds the size of the tree using the *size(x)* method suggested earlier, it takes $O(n)$ to find the size.

It then creates an empty new array of size n , $\Rightarrow O(1)$

A call to the method *traversal_save(x, arr, idx)* is called where $x = T.root$, $arr =$ the new empty array, $idx = 1$ since this is the first call.

Upon termination of the method, *arr* is altered and now the keys of the tree are stored in the array, the run-time of the method is $O(n)$.

Now a loop responsible for printing the keys iterates over the array by having an index responsible to the start of the array and using $size - index + 1$ we can have access to the appropriate value at the end of the array, we can simplify this by imagining we have to indexes, one at the start and one at the end, moving together until they meet or the first surpasses the second. In each iteration, the value stored in the array at “end index” is printed, and then the value stored at the “start index” is printed, since the array is sorted, the result is a *max_min* print of the keys.

The run-time complexity of the loop is $O(n)$, resulting in a total of $O(n)$ for executing the algorithm. ■

Part 2:

In this question, you are asked to implement the procedure $K_Nearest(A, x, k)$ that takes as an input an array A that contains n numbers, an additional number $x \in R$ and a natural number $1 \leq k \leq n$ and returns the k nearest numbers to x of all the numbers in the array A . The procedure must have a run-time of $O(n)$. When needed, you may break equality randomly. Provide pseudo-code, explanation on how the algorithm works and its correctness.

Solution:

Before suggesting the implementation, we know based on what was taught in the tutorials that the i th order statistic of n distinct numbers in an array can be achieved using a procedure that was taught $Select(A, i)$. This procedure returns the i th element in the sorted version of the array A , the run-time of this method is $O(n)$ based on what we were taught.

In order to achieve what was asked in the question, we will need a few things:

- 1) A distance array; this is an array that is of the same size as the input array where the following holds: $dist[i] = |A[i] - x|, \forall 1 \leq i \leq n$. The distance array will hold the "distance" of every value in A from x in the corresponding place.
- 2) Finding the i th order statistic of A in a linear run-time. This can be achieved using the $select(A, i)$ procedure taught in the lectures.

Here is the pseudo-code of the procedure $K_Nearest(A, x, k)$ and a procedure used for finding the distance array:

```
38 function createDistance(A, x){
39     dist = new arr[A.size]
40     for i = 1,...,n do
41         dist[i] = |A[i]-x|
42     end
43     return dist
44 }
45
46
47 function K_Nearest(A, x, k){
48     dist = createDistance(A,x)
49     B = new arr[k]
50     max_dist = Select(dist, k)
51     counter = 0
52     for i = 1,...,n do
53         if counter == k do
54             break
55         end
56         if(dist[i] <= max_dist) do
57             B[i] = A[i]
58             counter += 1
59         end
60     end
61     return B
62 }
```

Exactly like described above, the procedure finds the distance array, proceeds to find the value of the maximum distance by using the k th order statistic in A . Every value smaller or equal to that are relevant distances to us, thus we extract the corresponding value from A since we have the exact distance.

To further explain the idea, assume the following:

- $A = [3, 50, 6, 7, 11, 90, 2, 1]$
- $x = 4$
- $k = 3$

After terminating $K_Nearest(A, x, k)$ we expect the values $[3, 6, 2]$ to be returned since those are the k nearest values to x in A , here is a step by step on what the procedure did:

- 1) Create a new array representing the distances for each value in A from x ;
 $dist = |[3,50,6,7,11,90,2,1] - [x, x, x, x, x, x, x, x]| = [1,46,2,3,7,86,2,3]$
We can easily see that values that are distanced 1 or 2 are relevant values to us, let's see how the procedure found them:
- 2) Create a new empty array B of length k that will hold the k relevant values.
- 3) Finding the maximum distance, this is done by calling the method $Select(dist, k)$. This method returns the k th element in the ordered version of $dist$, thus in our case it will return 2 in our case, indicating that distances of 2 or less are relevant to us.
- 4) Loop over all the distances and extract the values of the element that are distanced max $dist$ at most, this can be done by iterating over the $dist$ array and extracting the relevant values from A or by iterating over A and for each element calculate the distance and add it B if it's relevant, either way this is done in $O(n)$.

The returned value is the array B that holds the relevant values, exactly how we expected.

Question 5:

Let $A, B \in \mathbb{R}^{n \times n}$, and assume that $n = 2^k$ where k is any natural number. Mark

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Where $A_{ij}, B_{ij} \in \mathbb{R}^{\frac{n}{2} \times \frac{n}{2}}$.

Part 1:

Remember that you can write matrix multiplication A and B in the following way:

$$A \cdot B = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & A_{11} \cdot B_{12} + A_{12} \cdot B_{22} \\ A_{21} \cdot B_{11} + A_{22} \cdot B_{21} & A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{bmatrix}$$

Use this fact in order to plan a recursive algorithm that calculates $A \cdot B$. You should provide pseudo-code. Describe the run-time complexity of the algorithm using recurrence relation and solve it to get an explicit asymptotic bound on the run-time complexity of the algorithm.

Solution:

We are asked to plan/implement an algorithm that recursively finds the result of a multiplication between two matrices, since the matrices are $\in \mathbb{R}^{n \times n}$ where n is a power of 2 and because the result matrix can be divided into **unrelated** parts, we will use the divide and conquer approach to implement the algorithm. Here is a brief explanation on what we will do:

- 1) We will split the matrix into four unrelated parts.
- 2) The result of matrix of the multiplication is also an $n \times n$ matrix that can also be separated into four unrelated parts, where each part can be established from two parts of each matrix that is involved in the multiplication.
- 3) Since all the parts are unrelated with one another, we will calculate each part recursively and then merge them together resulting in the final result of the multiplication.

We will need to introduce a few methods before proceeding with the algorithm:

```

65 function AddMatrices(A, B){
66     n = A.length
67     C = new arr[n][n]
68     for i = 1,...,n do
69         for j = 1,...,n do
70             C[i][j] = A[i][j] + B[i][j]
71         end
72     end
73 }
74
75 function connect(A, B, C, D){
76     n = A.length
77     E = new arr[2*n][2*n]
78     for i = 1,...,n do
79         for j = 1,...,n do
80             E[i][j] = A[i][j]
81             E[i][j + n] = B[i][j]
82             E[i + n][j] = C[i][j]
83             E[i + n][j + n] = D[i][j]
84         end
85     end
86     return E
87 }

```

$AddMatrices(A, B)$ takes as an input two matrices of size $n \times n$ and return a matrix representing the sum of A and B . It simply works by iterating over the two matrices and summing the numbers. $\Rightarrow O(n^2)$

$connect(A, B, C, D)$ takes as an input four matrices of size $n \times n$ and connects them resulting in a $2n \times 2n$ matrix, it does so by looping through a nested loop and assigning the values to their correct place in the new matrix. $\Rightarrow O(n^2)$

Here is the pseudo-code of the final method combining both methods above:

```

98 function MatrixMultiply(A, B){
99     n = A.length
100     if n == 1 do
101         return A[1][1] * B[1][1]
102     end
103     m = n/2
104     A11 = new empty arr[n/2][n/2]
105     A12 = new empty arr[n/2][n/2]
106     A21 = new empty arr[n/2][n/2]
107     A22 = new empty arr[n/2][n/2]
108     B11 = new empty arr[n/2][n/2]
109     B12 = new empty arr[n/2][n/2]
110     B21 = new empty arr[n/2][n/2]
111     B22 = new empty arr[n/2][n/2]
112     for i = 1,...,m do
113         for j = 1,...,m do
114             A11[i][j] = A[i][j]
115             B11[i][j] = B[i][j]
116             A12[i][j + m] = A[i][j + m]
117             B12[i][j + m] = B[i][j + m]
118             A21[i + m][j] = A[i + m][j]
119             B21[i + m][j] = B[i + m][j]
120             A22[i + m][j + m] = A[i + m][j + m]
121             B22[i + m][j + m] = B[i + m][j + m]
122         end
123     end
124     C11 = new empty arr[n/2][n/2]
125     C12 = new empty arr[n/2][n/2]
126     C21 = new empty arr[n/2][n/2]
127     C22 = new empty arr[n/2][n/2]
128     C11 = AddMatrices(MatrixMultiply(A11, B11), MatrixMultiply(A12, B21))
129     C12 = AddMatrices(MatrixMultiply(A11, B12), MatrixMultiply(A12, B22))
130     C21 = AddMatrices(MatrixMultiply(A21, B11), MatrixMultiply(A22, B21))
131     C22 = AddMatrices(MatrixMultiply(A21, B12), MatrixMultiply(A22, B22))
132     C = connect(C11, C12, C21, C22)
133     return C
134 }

```

Lines 92-93 is the base-case of the recursion, lines 96-103 initiates new empty matrices for dividing purposes of the problem, until now we have done $O(1)$ operations.

Nested loop lines 104-115 divides the original matrices into four parts. $O(n^2)$.

Lines 116-119 initiates four new empty matrices. $O(n^2)$.

Lines 121-124 are recursive calls to the function each where the input is 4 times smaller than the original call.

Line 126 connects the four results, resulting in a $n \times n$ matrix that is returned that constitutes the multiplication of the original matrices. $O(n^2)$

Time-complexity:

The run time of the algorithm can be described in the following way:

$$T(n) = O(n^2) + 8 \cdot T\left(\frac{n}{2}\right) + O(n^2) = 8 \cdot T\left(\frac{n}{2}\right) + 2 \cdot O(n^2) = 8 \cdot T\left(\frac{n}{2}\right) + O(n^2)$$

Let $a = 8, b = 2, f(n) = O(n^{\log_b a - \epsilon}) = O(n^{\log_2 8 - \epsilon}) = O(n^{3 - \epsilon})$, By choosing $\epsilon = 1$ we get: $f(n) = O(n^2)$.

$$\Rightarrow T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n).$$

According to the master theorem, this fits case 1 $\Rightarrow T(n) = \Theta(n^{\log_b a}) = \Theta(n^3)$. ■

Part 2:

It turns out, it is possible to calculate the multiplication these matrices in a more efficient way. Define the following matrices:

$$\begin{array}{llll} S_1 = B_{12} - B_{22} & S_2 = A_{11} + A_{12} & S_3 = A_{21} + A_{22} & S_4 = B_{21} - B_{11} \\ S_5 = A_{11} + A_{22} & S_6 = B_{11} + B_{22} & S_7 = A_{12} - A_{22} & S_8 = B_{21} + B_{22} \\ S_9 = A_{11} - A_{21} & S_{10} = B_{11} + B_{12} & & \end{array}$$

$$\begin{array}{llll} P_1 = A_{11} \cdot S_1 & P_2 = S_2 \cdot B_{22} & P_3 = S_3 \cdot B_{11} & P_4 = A_{22} \cdot S_4 \\ P_5 = S_5 \cdot S_6 & P_6 = S_7 \cdot S_8 & P_7 = S_9 \cdot S_{10} & \end{array}$$

$$\text{Show that } A \cdot B = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_5 + P_1 - P_3 - P_7 \end{bmatrix}$$

Use this to plan a recursive algorithm that calculates $A \cdot B$. You should provide pseudo-code. Describe the complexity of the algorithm using a recursion relation and solve it to get an explicit asymptotic bound on the run-time complexity of the algorithm. Compare the result with the result from part 1.

Solution:

Before suggesting an algorithm, let's prove that:

$$\begin{aligned} A \cdot B &= \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_5 + P_1 - P_3 - P_7 \end{bmatrix} \\ \text{To do that, let's mark } &\begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_5 + P_1 - P_3 - P_7 \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}. \\ C_{11} = P_5 + P_4 - P_2 + P_6 &= \underset{\text{assign the appropriate values}}{S_5 \cdot S_6 + A_{22} \cdot S_4 - S_2 \cdot B_{22} + S_7 \cdot S_8} \\ &= \underset{\text{assign the appropriate values}}{(A_{11} + A_{22}) \cdot (B_{11} + B_{22}) + A_{22} \cdot (B_{21} - B_{11}) - (A_{11} + A_{12}) \cdot B_{22} + (A_{12} - A_{22}) \cdot (B_{21} + B_{22})} = \\ &= A_{11}B_{11} + A_{11}B_{22} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} + A_{22}B_{21} - A_{22}B_{11} - A_{11}B_{22} - A_{12}B_{22} \\ &\quad + A_{12}B_{21} + A_{12}B_{22} - A_{22}B_{21} - A_{22}B_{22} = A_{11}B_{11} + A_{12}B_{21} \blacksquare \\ C_{12} = P_1 + P_2 &= \underset{\text{assign the appropriate values}}{A_{11} \cdot S_1 + S_2 \cdot B_{22}} \underset{\text{assign the appropriate values}}{=} \\ &= A_{11} \cdot (B_{12} - B_{22}) + (A_{11} + A_{12}) \cdot B_{22} = A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{22} + A_{12}B_{22} \\ &= A_{11}B_{12} + A_{12}B_{22} \blacksquare \\ C_{21} = P_3 + P_4 &= \underset{\text{assign the appropriate values}}{S_3 \cdot B_{11} + A_{22} \cdot S_4} \underset{\text{assign the appropriate values}}{=} \\ &= (A_{21} + A_{22}) \cdot B_{11} + A_{22} \cdot (B_{21} - B_{11}) = A_{21}B_{11} + A_{22}B_{11} + A_{22}B_{21} - A_{22}B_{11} \\ &= A_{21}B_{11} + A_{22}B_{21} \blacksquare \\ C_{22} = P_5 + P_1 - P_3 - P_7 &= \underset{\text{assign the appropriate values}}{S_5 \cdot S_6 + A_{11} \cdot S_1 - S_3 \cdot B_{11} - S_9 \cdot S_{10}} \\ &= \underset{\text{assign the appropriate values}}{(A_{11} + A_{22})(B_{11} + B_{22}) + A_{11} \cdot (B_{12} - B_{22})} \\ &\quad - (A_{21} + A_{22})B_{11} - (A_{11} - A_{21})(B_{11} + B_{12}) \\ &= A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} + A_{11}B_{12} - A_{11}B_{22} - A_{21}B_{11} \\ &\quad - A_{22}B_{11} - A_{11}B_{12} - A_{11}B_{12} + A_{21}B_{11} + A_{21}B_{12} = A_{21}B_{12} + A_{22}B_{22} \blacksquare \end{aligned}$$

The suggested equation gives the correct result, hence, we can proceed and suggest a recursive algorithm that solves the multiplication of two matrices based on what was proven above.

The algorithm we will suggest extracts $A_{ij}, B_{ij}, \forall i, j \in \{1, 2\}$ from A and B , exactly like we did in the previous part. $O(n^2)$

It then calculates $S_i, \forall i = 1, \dots, 10$ based on the given values of S_i . $O(n^2)$

It then calculates $P_i, \forall i = 1, \dots, 7$, based on the given values of P_i . Because the values of

P_i is the result of multiplication between matrices, the value will be calculated recursively.

When done calculating P_i , a new matrix will be assembled and returned, it is assembled from adding some $P_{i's}$ and then combining the result into one matrix.

We will use the extra methods introduced in the previous part and a new method that subtracts two matrices, this method works similarly to the method *AddMatrices* that was introduced in the previous part.

```

132 function SubtractMatrices(A, B){
133     n = A.length
134     C = new arr[n][n]
135     for i = 1,...,n do
136         for j = 1,...,n do
137             C[i][j] = A[i][j] - B[i][j]
138         end
139     end
140 }

```

Here is the pseudo-code:

```

144 function OptimizedMatrixMultiply(A, B){
145     n = A.length
146     if n == 1 do
147         return A[1][1] * B[1][1]
148     end
149     m = n/2
150     A11 = new empty arr[n/2][n/2]
151     A12 = new empty arr[n/2][n/2]
152     A21 = new empty arr[n/2][n/2]
153     A22 = new empty arr[n/2][n/2]
154     B11 = new empty arr[n/2][n/2]
155     B12 = new empty arr[n/2][n/2]
156     B21 = new empty arr[n/2][n/2]
157     B22 = new empty arr[n/2][n/2]
158     for i = 1,...,m do
159         for j = 1,...,m do
160             A11[i][j] = A[i][j]
161             B11[i][j] = B[i][j]
162             A12[i][j + m] = A[i][j + m]
163             B12[i][j + m] = B[i][j + m]
164             A21[i + m][j] = A[i + m][j]
165             B21[i + m][j] = B[i + m][j]
166             A22[i + m][j + m] = A[i + m][j + m]
167             B22[i + m][j + m] = B[i + m][j + m]
168         end
169     end
170     S1 = SubtractMatrices(B12, B22)
171     S2 = AddMatrices(A11, A12)
172     S3 = AddMatrices(A21, A22)
173     S4 = SubtractMatrices(B21, B11)
174     S5 = AddMatrices(A11, A22)
175     S6 = AddMatrices(B11, B22)
176     S7 = SubtractMatrices(A12, A22)
177     S8 = AddMatrices(B21, B22)
178     S9 = SubtractMatrices(A11, A21)
179     S10 = AddMatrices(B11, B12)
180     P1 = OptimizedMatrixMultiply(A11, S1)
181     P2 = OptimizedMatrixMultiply(S2, B22)
182     P3 = OptimizedMatrixMultiply(S3, B11)
183     P4 = OptimizedMatrixMultiply(A22, S4)
184     P5 = OptimizedMatrixMultiply(S5, S6)
185     P6 = OptimizedMatrixMultiply(S7, S8)
186     P7 = OptimizedMatrixMultiply(S9, S10)
187     C11 = SubtractMatrices(AddMatrices(P5, P4), AddMatrices(P2, P6))
188     C12 = AddMatrices(P1, P2)
189     C21 = AddMatrices(P3, P4)
190     C22 = SubtractMatrices(AddMatrices(P5, P1), SubtractMatrices(P3, P7))
191     C = connect(C11, C12, C21, C22)
192     return C
193 }

```

Lines 145-169 are exactly the same as before, these lines are responsible to dividing the matrices into 4 parts each, this is done in $O(n^2)$.

Lines 170-179 are responsible for evaluating all the $S_{i's}$, this is done in $O(n^2)$ also.

Lines 180-186 are responsible for evaluating all the $P_{i's}$, this is done by recursively calling the method on smaller inputs, hence the run-time is $T\left(\frac{n}{2}\right)$ for each call.

Lines 187-190 are responsible for evaluating all the C_{ij} , this is done by adding and subtracting the matrices. $O(n^2)$.

Line 191 is responsible for connecting the results, $O(n^2)$.

The run-time of the method can be described using a recurrence relation:

$$T(n) = O(n^2) + O(n^2) + 7 \cdot T\left(\frac{n}{2}\right) + O(n^2) + O(n^2) = 7 \cdot T\left(\frac{n}{2}\right) + O(n^2).$$

$$\text{Let } a = 7, b = 2, f(n) = O(n^{\log_b a - \epsilon}) = O(n^{\log_2 7 - \epsilon}) \stackrel{\log_2 7 \approx 2.807}{=} O(n^{2.807 - \epsilon}).$$

By choosing $\epsilon = \log_2 7 - 2 \approx 0.807$, we get $f(n) = O(n^2)$. According to the master theorem, this is case 1, and we get that $T(n) = \Theta(n^{\log_b a}) \approx \Theta(n^{2.807})$. ■

Comparing:

When comparing the run-time complexity of this method with the previous method, we can easily see that this method is more efficient since it's run-time is better than the previous one, although it is not better by much, but when running the algorithm for large enough input we will see the difference.